MLflow — Project Recipe Card

Tracking · logging · registry (aliases) · signatures · autolog · HPO · evaluation · serving. Deep companion: `mlflow_cheatsheet.md` · Stack: `MLflow ≥2.12` · `Python 3.10+` · Modern API (aliases, not stages; `name=` not `artifact_path=`)

◆ Universal recipe (every framework, every run)

1. **SETUP** (process-level) — `set_tracking_uri` + `set_experiment`
2. **INSIDE run** — `with mlflow.start_run() as run:`
3. **log_params** — inputs (hyperparams). ONE-SHOT. str/int/float/bool.
4. **do the work** — train / fit / build
5. **log_metric × N** — outputs. Float-only. `step=` = time series.
6. **log_model** — artifact + **signature** + **input_example** + (optional) register
7. **AFTER run** — `set_registered_model_alias` tags the new version (@candidate / @production)

Registry ops live OUTSIDE the run. Aliases act on the registry, not the run — inside works mechanically but is semantically wrong.

◆ Params vs metrics · Modern API changes

PARAMS VS METRICS

	log_param	log_metric
Captures	INPUTS (knobs)	OUTPUTS (results)
Examples	lr, hidden, seed	train_loss, val_auc
Mutable?	NO (re-set raises)	YES (step= = series)
Type	str/int/float/bool	float only

MODERN API — WHAT CHANGED

Legacy (don't)	Modern (do)
artifact_path="model"	name="model"
transition_model_version_stage (Staging)	set_registered_model_alias("…", "candidate", v)
get_latest_versions(stages=["Production"])	get_model_version_by_alias("…", "production")
URI models:/name/Production	URI models:/name@production
cloudpickle (sklearn default)	skops (safer; no code-exec)

◆ Starter · A: setup + minimal run

```
import mlflow
from mlflow import MlflowClient
from mlflow.models import infer_signature

mlflow.set_tracking_uri("sqlite:///mlflow.db")       # or "http://mlflow:5000" in prod
mlflow.set_experiment("my-experiment")

with mlflow.start_run(run_name="lr=1e-3") as run:
    mlflow.log_params({"lr": 1e-3, "hidden": "[64,64]", "seed": 42})
    model, history = train(...)
    for ep, (tr, va) in enumerate(zip(history["train"], history["val"])):
        mlflow.log_metric("train_loss", tr, step=ep)
        mlflow.log_metric("val_loss",   va, step=ep)
    sig = infer_signature(X_tr[:5], model.predict(X_tr[:5]))
    mi = mlflow.sklearn.log_model(
        sk_model=model, name="model",
        registered_model_name="my-model",
        signature=sig, input_example=X_tr[:5])
    run_id = run.info.run_id
```

◆ Starter · B: registry alias + hygiene

```
# 7+8. AFTER the run — registry ops act on the registry, NOT the run
client = MlflowClient()
v = mi.registered_model_version                      # no round-trip needed
client.set_registered_model_alias("my-model", "candidate", v)

# Hygiene — in code, never UI (reproducibility):
client.update_registered_model("my-model",
    description="Trade outcome classifier — GBM, hourly retrain.")
client.update_model_version("my-model", v,
    description=f"n={len(X_tr)}, val_auc={best_auc:.3f}")
for k, val in {"owner":"team-quant", "framework":"sklearn",
               "task":"classification"}.items():
    client.set_registered_model_tag("my-model", k, val)
for k, val in {"git_commit":"abc1234",
"validation_status":"pending"}.items():
    client.set_model_version_tag("my-model", v, k, val)

# Promote (after eval gate — see CI section)
client.set_registered_model_alias("my-model", "production", v)
client.delete_registered_model_alias("my-model", "candidate")
```

● Tracking URIs + experiments

```
mlflow.set_tracking_uri("./mlruns")                  # file-based — dev only
mlflow.set_tracking_uri("sqlite:///mlflow.db")       # local sqlite — single user
mlflow.set_tracking_uri("postgresql://...")          # prod meta store
mlflow.set_tracking_uri("http://mlflow:5000")        # remote tracking server

# Server: mlflow server --backend-store-uri postgresql://...
#                       --default-artifact-root s3://bucket/
mlflow

mlflow.set_experiment("trading/classifier")          # creates if missing
exp = mlflow.get_experiment_by_name("trading/classifier")
print(exp.experiment_id, exp.artifact_location)
```

Env var	Effect
MLFLOW_TRACKING_URI	overrides set_tracking_uri
MLFLOW_EXPERIMENT_NAME	overrides set_experiment
MLFLOW_TRACKING_TOKEN	bearer auth on managed servers
MLFLOW_S3_ENDPOINT_URL	S3-compatible artifact store

● Runs — the core context

```
with mlflow.start_run(run_name="my-run",
                      tags={"git_commit": git_sha, "owner": "x"}) as run:
    run_id = run.info.run_id
    ...

# Resume a run (e.g. add metrics after training restart):
with mlflow.start_run(run_id="abc123"):
    mlflow.log_metric("post_hoc_score", 0.91)

# Active run access:
run = mlflow.active_run()
mlflow.set_tag("notes", "investigated overfitting at epoch 30")

# Search:
runs = mlflow.search_runs(experiment_names=["my-exp"],
                          filter_string="metrics.val_auc > 0.85",
                          order_by=["metrics.val_auc DESC"],
                          max_results=10)
```

Always set `run_name` — makes the UI scannable. Tags are mutable; params aren't.

● log_params / log_metric / log_artifact

```
mlflow.log_param("lr", 1e-3)
mlflow.log_params({"lr": 1e-3, "hidden": "[64,64]", "seed": 42})  # batched

mlflow.log_metric("train_loss", 0.42)
mlflow.log_metric("train_loss", 0.38, step=1)        # time series
mlflow.log_metrics({"acc": 0.9, "f1": 0.87}, step=epoch)          # batched

mlflow.log_artifact("plots/loss_curve.png")                  # single file
mlflow.log_artifacts("./plots", artifact_path="plots")            # whole dir

# Figures inline:
import matplotlib.pyplot as plt
fig, ax = plt.subplots(); ax.plot(losses)
mlflow.log_figure(fig, "loss_curve.png")

# Text + dict (JSON):
mlflow.log_text(report, "classification_report.txt")
mlflow.log_dict({"feature_cols": list(X.columns)}, "schema.json")
```

Non-string params: stringify lists/dicts BEFORE logging — `log_params` coerces but the round-trip loses type. Use `str([64,64])` not `[64,64]`.

▲ Model logging — sklearn (default)

```
from sklearn.linear_model import LogisticRegression
from mlflow.models import infer_signature

pipe = Pipeline([("sc", StandardScaler()), ("clf", LogisticRegression())])
pipe.fit(X_tr, y_tr)

sig = infer_signature(X_tr, pipe.predict(X_tr))      # input + output schema
mi = mlflow.sklearn.log_model(                       # capture ModelInfo (see Starter B)
    sk_model=pipe,
    name="model",                                    # NOT artifact_path=
    registered_model_name="my-classifier",
    signature=sig,
    input_example=X_tr.iloc[:5],                     # 1-5 rows for UI preview
    serialization_format="skops",                    # safer than cloudpickle (arbitrary-code)
    pip_requirements=["scikit-learn==1.5.0"],
)
# mi.registered_model_version is the new version — use directly, no round-trip!
```

skops serialisation only stores trusted types (no arbitrary pickle code-exec on load). Always pass `serialization_format="skops"` explicitly — safer than cloudpickle.

▲ Model logging — PyTorch

```
import mlflow.pytorch

sig = infer_signature(X_tr[:5].numpy(), model(X_tr[:5]).detach().numpy())
mi = mlflow.pytorch.log_model(            # capture ModelInfo — see Starter B
    pytorch_model=model,
    name="model",
    registered_model_name="my-nn",
    signature=sig,
    input_example=X_tr[:5].numpy(),
    pip_requirements=[
        "torch==2.5.0+cu121",                # CUDA pin via PyTorch extra-index-url
        "--extra-index-url https://download.pytorch.org/whl/cu121",
    ],
)
```

Pin Python AND torch+CUDA. `torch==2.5.0` alone resolves CPU-only on PyPI — for GPU serving append `+cu121` with the PyTorch index URL. Without Python pin, training-3.11 → serving-3.12 mismatch breaks pickle. Use

python_version="3.11" kwarg or let MLflow auto-emit python_env.yaml from the current interpreter.

▲ XGBoost / LightGBM / CatBoost

```
import mlflow.lightgbm
import lightgbm as lgb

m = lgb.LGBMClassifier(n_estimators=500, learning_rate=0.05)
m.fit(X_tr, y_tr,
      eval_set=[(X_va, y_va)], eval_metric="auc",
      callbacks=[lgb.early_stopping(50), lgb.log_evaluation(0)])

mlflow.lightgbm.log_model(
    lgb_model=m,
    name="model",
    signature=infer_signature(X_tr, m.predict(X_tr)),
    input_example=X_tr.iloc[:5],
    registered_model_name="my-gbm",
)

# Same shape: mlflow.xgboost.log_model(xgb_model=…),
mlflow.catboost.log_model(cb_model=…)
```

Autolog (§autolog) is especially nice for GBMs — captures per-tree metrics + feature importances automatically.

▲ Transformers / LangChain (LLMs)

```
# HuggingFace Transformers
import mlflow.transformers
from transformers import pipeline
hf_pipe = pipeline(
    "text-classification",
    model="distilbert-base-uncased-finetuned-sst-2-english")
mlflow.transformers.log_model(
    transformers_model=hf_pipe,
    name="model",
    task="text-classification",
    registered_model_name="sentiment",
    input_example={"inputs": ["MLflow is great", "I
disagree"]})  # dict portable

# LangChain / LangGraph
import mlflow.langchain
mlflow.langchain.log_model(
    lc_model=chain,                  # or graph (LangGraph)
    name="model",
    input_example={"question": "hi"})
```

▲ Custom pyfunc model (anything not a flavour)

```
import mlflow.pyfunc

class TwoModelEnsemble(mlflow.pyfunc.PythonModel):
    def load_context(self, context):
        # Called ONCE when the model is loaded for inference
        import joblib
        self.a = joblib.load(context.artifacts["model_a"])
        self.b = joblib.load(context.artifacts["model_b"])

    def predict(self, context, model_input, params=None):
        return 0.5 * self.a.predict(model_input) + 0.5 *
self.b.predict(model_input)

mlflow.pyfunc.log_model(
    name="ensemble",
    python_model=TwoModelEnsemble(),
    artifacts={"model_a": "models:/a@production",
               "model_b": "models:/b@production"},
    pip_requirements=["scikit-learn==1.5.0", "joblib==1.4"],
    input_example=X_tr.iloc[:5],
    signature=infer_signature(X_tr, np.zeros(len(X_tr))),
)
```

Use pyfunc for: ensembles, custom pre/post-processing, multi-model serving, any non-standard flavour.

◆ Signatures + input examples (ALWAYS)

```
from mlflow.models import infer_signature, ModelSignature
from mlflow.types import Schema, ColSpec, DataType, TensorSpec

# Easy way — infer from a sample
sig = infer_signature(X_tr, model.predict(X_tr))   # or
pd.DataFrame / np.ndarray / dict

# Explicit way (recommended for production stability)
sig = ModelSignature(
    inputs=Schema([
        ColSpec(DataType.double, "age"),
        ColSpec(DataType.string, "region"),
    ]),
    outputs=Schema([ColSpec(DataType.double, "score")]),
)

# Input example for the UI + serving tester
input_example = X_tr.iloc[:5]    # pandas DataFrame /
np.ndarray / dict
```

Always pass `signature=` AND `input_example=`. Without them: no schema validation at serve-time, no UI preview, no automatic curl-test snippet. Cheap insurance.

◆ Model Registry — alias workflow (3.x)

```
client = MlflowClient()

# 1. Create / update registered model + version (auto with
registered_model_name=)
mv = client.create_model_version(
    name="my-model",
    source=f"runs:/{run_id}/model",     # or models:/
from_other_model
    run_id=run_id,
)

# 2. ALIASES (modern API — stages deprecated since 2.9, removed
in 3.x)
client.set_registered_model_alias("my-model", "candidate",
mv.version)        # newly trained
client.set_registered_model_alias("my-model", "production",
"7")              # serving NOW
# A/B test pattern (champion vs challenger point at DIFFERENT
versions):
client.set_registered_model_alias("my-model", "champion",
"7")               # current best
client.set_registered_model_alias("my-model", "challenger",
mv.version)       # new candidate

# 3. LOOKUP by alias
prod = client.get_model_version_by_alias("my-model",
"production")
print(prod.version, prod.run_id, prod.tags)

# 4. Cleanup
client.delete_registered_model_alias("my-model", "candidate")
client.delete_model_version("my-model", "5")
client.delete_registered_model("my-model")           # cascades
versions
```

URI form	Resolves to
models:/my-model@production	version aliased "production" (RECOMMENDED)
models:/my-model/5	specific version 5 (pinned)
models:/my-model/latest	latest version (avoid in prod — moves)
runs:/{run_id}/model	artifact within a run (unregistered)

Stages are dead. `transition_model_version_stage(...)` emits a deprecation warning. Aliases are the future — multi-alias (champion + challenger), case-insensitive, free-form.

● Loading models

```
# Generic pyfunc loader (works for ALL flavours)
import mlflow.pyfunc
m = mlflow.pyfunc.load_model("models:/my-model@production")
y = m.predict(X_new)                          # pandas / numpy /
dict in

# Flavour-specific loader (returns the original framework
object)
m = mlflow.sklearn.load_model("models:/my-model@production")   #
→ sklearn estimator
m = mlflow.pytorch.load_model("models:/my-nn@production")
# → torch.nn.Module
m = mlflow.lightgbm.load_model("models:/my-gbm@production")
# → LGBMClassifier

# From a run (not registered) / download artifacts
m = mlflow.pyfunc.load_model(f"runs:/{run_id}/model")
mlflow.artifacts.download_artifacts(
    artifact_uri="models:/my-model@production", dst_path="./
local")
```

Use **pyfunc** in serving (uniform API); use the **flavour** loader for fine-tuning / introspection.

● Autologging — when to use, when to skip

```
mlflow.autolog()                              # turns on ALL
installed flavours
mlflow.sklearn.autolog()                      # framework-
specific (preferred)
mlflow.xgboost.autolog(log_models=False)      # skip model
artifact (just metrics)
mlflow.lightgbm.autolog()
mlflow.pytorch.autolog()                      # PyTorch
Lightning + manual hooks

# Then just train normally — params, metrics, model logged
automatically:
gs = GridSearchCV(...).fit(X_tr, y_tr)        # one parent run +
nested per CV fold
```

Use autolog when	SKIP autolog when
Sklearn / xgb / lgb / pytorch-lightning	Custom training loops (PyTorch raw)
You want zero-friction tracking	You need precise control over logged metrics
HPO with GridSearchCV / Optuna	Multi-stage pipelines you want as one run

Autolog can log **huge** artifacts (full GridSearch state). Use `log_models=False` + selective `log_metric` calls if size matters.

▲ Pattern — HPO with nested runs

```
import optuna

def objective(trial):
    params = {"lr":    trial.suggest_float("lr", 1e-4, 1e-1,
log=True),
              "depth": trial.suggest_int("depth", 3, 10)}
    with mlflow.start_run(nested=True, run_name=f"trial-
{trial.number}"):
        mlflow.log_params(params)
        model = train(**params)
        val_loss = evaluate(model, X_va, y_va)
        mlflow.log_metric("val_loss", val_loss)
        return val_loss

with mlflow.start_run(run_name="hpo-study"):
    study = optuna.create_study(direction="minimize",

sampler=optuna.samplers.TPESampler(seed=42))
    study.optimize(objective, n_trials=50)
    mlflow.log_params(study.best_params)
    mlflow.log_metric("best_val_loss", study.best_value)
    # Re-train on best params, log_model on the OUTER run only
```

Outer run = study summary; nested runs = individual trials. The UI groups them automatically.

▲ Pattern — CV as a metric series

```
from sklearn.model_selection import KFold

with mlflow.start_run(run_name="kfold-cv"):
```

```
    mlflow.log_params({"model": "lgbm", "n_estimators": 500})
    kf = KFold(5, shuffle=True, random_state=42)
    aucs = []
    for k, (tr, va) in enumerate(kf.split(X)):
        model = fit(X.iloc[tr], y.iloc[tr])
        auc = roc_auc_score(y.iloc[va],
model.predict_proba(X.iloc[va])[:, 1])
        mlflow.log_metric("fold_auc", auc, step=k)    # step=
turns it into a series
        aucs.append(auc)
    mlflow.log_metric("cv_auc_mean", np.mean(aucs))
    mlflow.log_metric("cv_auc_std",  np.std(aucs))
```

▲ Dataset tracking (`mlflow.data`)

```
from mlflow.data import from_pandas

ds = from_pandas(df_train, source="s3://bucket/train.parquet",
                 targets="y", name="train_v2")
with mlflow.start_run():
    mlflow.log_input(ds, context="training")
    mlflow.log_input(from_pandas(df_val, name="val_v2"),
context="validation")
    # ...

# Later: see exactly which data went into a run (UI → "Datasets"
tab).
# Use a content hash in the dataset name (e.g.
"train_sha:abc123") for
# guaranteed lineage — bare versioned names drift silently.
```

▲ Model evaluation (`mlflow.evaluate`)

```
# One-call evaluation: metrics + plots + explainability (shap)
result = mlflow.evaluate(
    model="models:/my-model@candidate",         # or a callable
    data=eval_df,                               # pd.DataFrame
with target col
    targets="y_true",
    model_type="classifier",                    # "regressor" /
"question-answering" / "text-summarization"
    evaluators=["default"],                     # adds ROC, PR,
confusion matrix, SHAP
    evaluator_config={"explainability_nsamples": 500},
)
print(result.metrics)                  # auto-logged to current/
active run
print(result.artifacts)                # plots + explainer
artifacts
```

Use as a **promotion gate**: run evaluate on a frozen test set; gate `candidate → production` on threshold (e.g. `val_auc > 0.85`).

◆ Serving (local + container)

```
# Quick local server (REST):
mlflow models serve -m "models:/my-model@production" -p 5000 --
no-conda
# POST /invocations  body: {"dataframe_split": {...}} or
{"instances": [...]}
curl -X POST http://localhost:5000/invocations \
     -H 'Content-Type: application/json' \
     -d '{"dataframe_split": {"columns":["age","income"],
"data":[[42, 80000]]}}'

# Build a Docker image:
mlflow models build-docker -m "models:/my-model@production" -n
my-model:latest

# Generate Dockerfile only (CI / customise):
mlflow models generate-dockerfile -m "models:/my-
model@production" -d ./dockerfile/

# Production deploy targets (managed):
# mlflow deployments create -t sagemaker -m models:/m@production
--name prod
# mlflow deployments create -t databricks -m models:/
m@production --name prod
```

For production: load via `mlflow.pyfunc.load_model` inside a FastAPI app (more control over auth, observability, batching) — `mlflow models serve` is best for quick local tests.

◆ CI / promotion gate (canonical)

```
# promote.py — run in CI after every train job
import mlflow
from mlflow import MlflowClient
client = MlflowClient()

def candidate_passes(model_uri: str, test_df, threshold: float =
0.85) -> bool:
    r = mlflow.evaluate(model=model_uri, data=test_df,
targets="y",
                        model_type="classifier",
evaluators=["default"])
    return r.metrics["roc_auc"] > threshold

cand = client.get_model_version_by_alias("my-model",
"candidate")
uri = f"models:/my-model@candidate"
if not candidate_passes(uri, test_df):
    client.set_model_version_tag("my-model", cand.version,
"validation_status", "failed")
    raise SystemExit(1)                         # fail CI

# Guarded swap — refuse to overwrite a newer @production
(concurrent CI race)
try:
    cur_prod = client.get_model_version_by_alias("my-model",
"production")
    assert int(cand.version) > int(cur_prod.version), \
        f"refusing to set @production={cand.version} over newer
v{cur_prod.version}"
except mlflow.exceptions.RestException:
    pass                                        # no current
@production — first promotion

# Audit trail BEFORE the swap (so log entry exists even if swap
fails)
client.set_model_version_tag("my-model", cand.version,
"validation_status", "passed")
client.set_model_version_tag("my-model", cand.version,
"promoted_by_pr",
                             os.environ.get("GITHUB_PR",
"manual"))
client.set_model_version_tag("my-model", cand.version,
"promoted_at",

datetime.now(timezone.utc).isoformat())

client.set_registered_model_alias("my-model", "production",
cand.version)
client.delete_registered_model_alias("my-model", "candidate")
```

Rollback: `prev = search_model_versions("name='m' AND tags.validation_status='passed'", order_by=["version_number DESC"], max_results=2)[1]` → `set_registered_model_alias("m", "production", prev.version)`. One-liner; keep `rollback.py` in the repo.

● Configuration that earns its place

- `MLFLOW_TRACKING_URI` — set once in env; don't hard-code in scripts
- `MLFLOW_EXPERIMENT_NAME` — folder grouping; overrides `set_experiment`
- `MLFLOW_S3_ENDPOINT_URL` — for MinIO / R2 / non-AWS S3
- `MLFLOW_REGISTRY_URI` — split metadata + registry stores if needed
- `MLFLOW_HTTP_REQUEST_TIMEOUT` — default 120s, raise for slow networks
- `MLFLOW_DISABLE_INSECURE_REQUEST_WARNING=1` — silence for self-signed certs

Server boot: `mlflow server --backend-store-uri postgresql://… --default-artifact-root s3://…` + auth via reverse proxy (Caddy / Traefik).

▲ Top anti-patterns (6-month memory)

- **Stages** (Staging/Production) — deprecated. Use **aliases** (@production, @candidate).
- **`artifact_path=`** — deprecated. Use `name=`.
- **No signature / no input_example** — no schema validation at serve-time, no UI preview.
- **Re-setting a param** in the same run → raises. Use tags or restart the run.
- **Logging dict/list as param without `str()`** → silently coerced; round-trip loses type.
- **Registry alias INSIDE the run** — works mechanically but semantically wrong; aliases are registry ops.
- **UI clicks for hygiene** (description, tags) — not reproducible. Always in code.
- **cloudpickle for sklearn** in 2026 — arbitrary code-exec on load. Use `serialization_format="skops"`.
- **Autolog without `log_models=False`** on GridSearchCV → artifact bloat (full state per fold).
- **No `pip_requirements=`** on `log_model` → reproducibility roulette. Pin exact framework versions.
- **Logging from inside a Spark UDF** → distributed write conflicts. Aggregate, then log on the driver.
- **`models:/.../latest`** in prod → moves under you. Pin to alias or version.
- **SQLite tracking URI** for multi-process / concurrent training → corruption. Use Postgres in prod.
- **No experiment tags** → search-by-purpose fails (`tags.purpose='shadow-eval'`).
- **Skipping `mlflow.evaluate`** as a promotion gate → models promoted manually without proof.

◆ Quick-reference (recall in 5 seconds)

Task	Canonical pattern
Tracking URI	`mlflow.set_tracking_uri("postgresql://...")`
Experiment	`mlflow.set_experiment("trading/classifier")`
Start run	`with mlflow.start_run(run_name=..., tags={...}) as run:`
Nested run	`with mlflow.start_run(nested=True):`
Log params	`mlflow.log_params({"lr": 1e-3, "hidden": str([64,64])})`
Log metric series	`mlflow.log_metric("val_loss", v, step=epoch)`
Log artifact	`mlflow.log_artifact("plot.png")` / `log_figure(fig, "p.png")`
Log sklearn model	`mlflow.sklearn.log_model(sk_model=m, name="model", signature=sig, input_example=X[:5], registered_model_name="x")`
Log pytorch model	`mlflow.pytorch.log_model(pytorch_model=m, name="model", serialization_format="pt2")`
Custom pyfunc	subclass `mlflow.pyfunc.PythonModel` + `load_context` + `predict`
Signature	`infer_signature(X_tr, model.predict(X_tr))`
Autolog	`mlflow.sklearn.autolog()` (or `mlflow.autolog()`)
Set alias	`client.set_registered_model_alias("m", "production", v)`
Get by alias	`client.get_model_version_by_alias("m", "production")`
Load model	`mlflow.pyfunc.load_model("models:/m@production")`
Search runs	`mlflow.search_runs(experiment_names=[...], filter_string="metrics.auc > 0.8")`
Evaluate	`mlflow.evaluate(model=uri, data=df, targets="y", model_type="classifier")`
Dataset log	`mlflow.log_input(from_pandas(df, name="train_v2"), context="training")`
Serve local	`mlflow models serve -m "models:/m@production" -p 5000`
Build docker	`mlflow models build-docker -m "models:/m@production" -n my-model`
Promote gate	`mlflow.evaluate` → threshold → `set_registered_model_alias`